Homework 5

PSTAT 131/231 Elliott

Homework 5

For this assignment, we will be working with the file `"pokemon.csv"`, found in `/data`. The file is from Kaggle: https://www.kaggle.com/abcsds/pokemon.

The Pokémon franchise encompasses video games, TV shows, movies, books, and a card game. This data set was drawn from the video game series and contains statistics about 721 Pokémon, or "pocket monsters." In Pokémon games, the user plays as a trainer who collects, trades, and battles Pokémon to (a) collect all the Pokémon and (b) become the champion Pokémon trainer.

Each Pokémon has a primary type (some even have secondary types). Based on their type, a Pokémon is strong against some types, and vulnerable to others. (Think rock, paper, scissors.) A Fire-type Pokémon, for example, is vulnerable to Water-type Pokémon, but strong against Grass-type.

Figure 1: Fig 1. Vulpix, a Fire-type fox Pokémon from Generation 1 (also my favorite Pokémon!)

The goal of this assignment is to build a statistical learning model that can predict the **primary type** of a Pokémon based on its generation, legendary status, and six battle statistics. *This is an example of a* ***classification problem****, but these models can also be used for* ***regression problems****.*

Read in the file and familiarize yourself with the variables using `pokemon_codebook.txt`.

Exercise 1

Install and load the `janitor` package. Use its `clean_names()` function on the Pokémon data, and save the results to work with for the rest of the assignment. What happened to the data? Why do you think `clean_names()` is useful?

```
library(janitor)
pok_data <- read.csv("Pokemon.csv")
names(pok_data)
```

```
##  [1] "X."         "Name"       "Type.1"     "Type.2"     "Total"
##  [6] "HP"         "Attack"     "Defense"    "Sp..Atk"    "Sp..Def"
## [11] "Speed"      "Generation" "Legendary"
```

```
pok_clean_data <- pok_data %>% clean_names()
names(pok_clean_data)
```

```
##  [1] "x"          "name"       "type_1"     "type_2"     "total"
##  [6] "hp"         "attack"     "defense"    "sp_atk"     "sp_def"
## [11] "speed"      "generation" "legendary"
```

After using `clean_names()` function, the letters in column names were changed to lowercase letters and some special characters were changed in to "_". For example, the word"Name" was changed into "name", and "Type.1" was changed into "type_1".

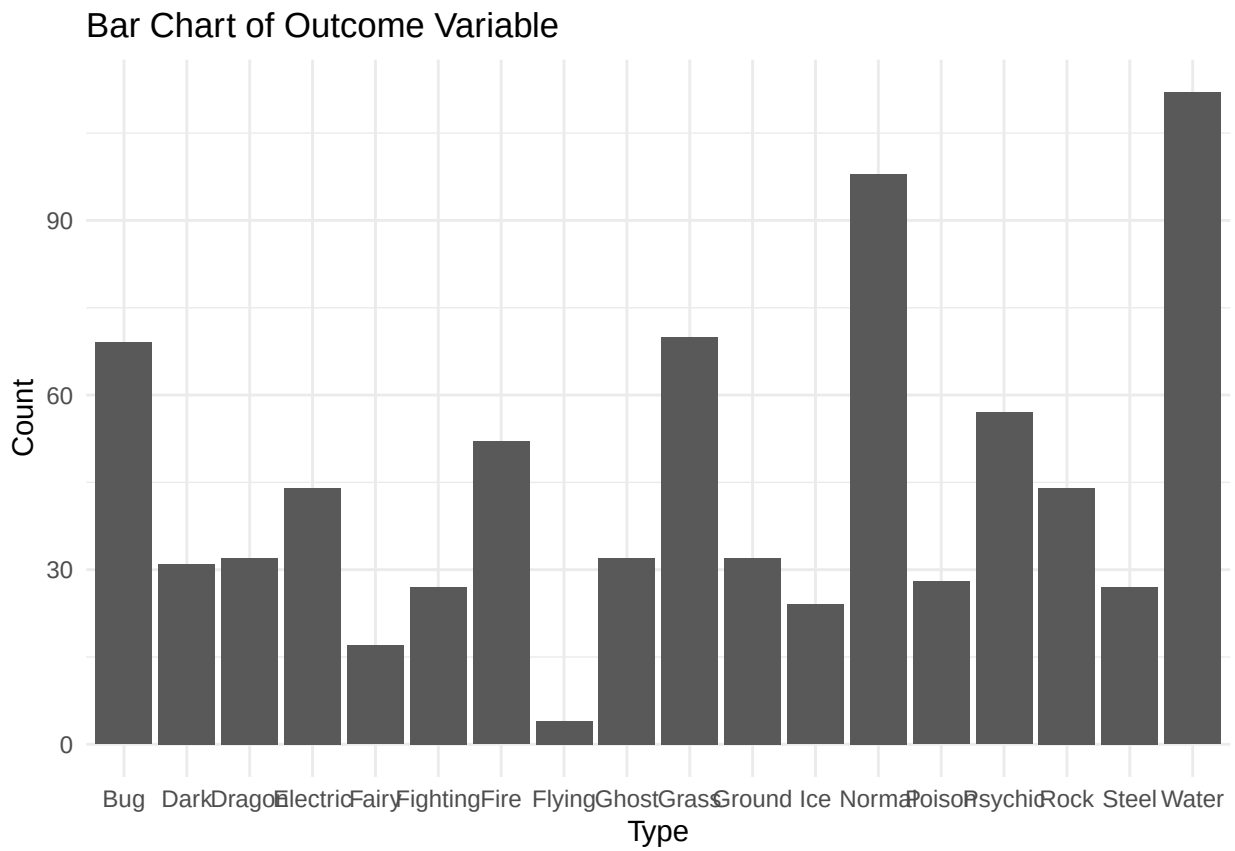
This function is useful because it standardizes the column names but did not change any data in the table. By doing this, all the variables' names become consistent and follow the same format, which can avoid some problems caused by space or some other special characters.

Exercise 2

Using the entire data set, create a bar chart of the outcome variable, `type_1`.

```
library(tidyverse)
library(forcats)
library(ggplot2)
library(tidymodels)
library(corrplot)
library(vip)
set.seed(131)
```

```
pok_clean_data %>% ggplot(aes(x = type_1)) +
  geom_bar() +
  labs(title = "Bar Chart of Outcome Variable",
       x = "Type",
       y = "Count") +
  theme_minimal()
```



How many classes of the outcome are there? Are there any Pokémon types with very few Pokémon? If so, which ones?

```
n_distinct(pok_clean_data$type_1)
```

```
## [1] 18
```

There are 18 classes of the outcome variable `type_1`. Type such as `Flying` has the lowest frequency, and types such as `Fairy`, `Ice`, `Steel`, and `Poison` appear less frequently than other variables.

For this assignment, we'll handle the rarer classes by grouping them, or "lumping them," together into an 'other' category. Using the `forcats` package, determine how to do this, and **lump all the other levels together except for the top 6 most frequent** (which are Bug, Fire, Grass, Normal, Water, and Psychic).

Convert `type_1` and `legendary` to factors.

```
pok_clean_data <- pok_clean_data %>%
  mutate(type_1 = fct_lump(type_1, n = 6),
         type_1 = as.factor(type_1),
         legendary = as.factor(legendary))
```

Exercise 3

Perform an initial split of the data. Stratify by the outcome variable. You can choose a proportion to use. Verify that your training and test sets have the desired number of observations.

Next, use v-fold cross-validation on the training set. Use 5 folds. Stratify the folds by `type_1` as well. *Hint: Look for a* ***`strata`*** *argument.*

```
data_split <- initial_split(
  pok_clean_data,
  prop = 0.75,
  strata = type_1
)

train <- training(data_split)
test <- testing(data_split)

nrow(train)
```

```
## [1] 599
```

```
nrow(test)
```

```
## [1] 201
```

```
cv_fold <- vfold_cv(
  train,
  v = 5,
  strata = type_1
)
```

Training set has 599 observations and testing set has 201 observations. $\frac{599}{201+599} = 0.74875 \approx 0.75$ and $\frac{201}{201+599} = 0.25125 \approx 0.25$, which corresponds to our data splitting proportion.

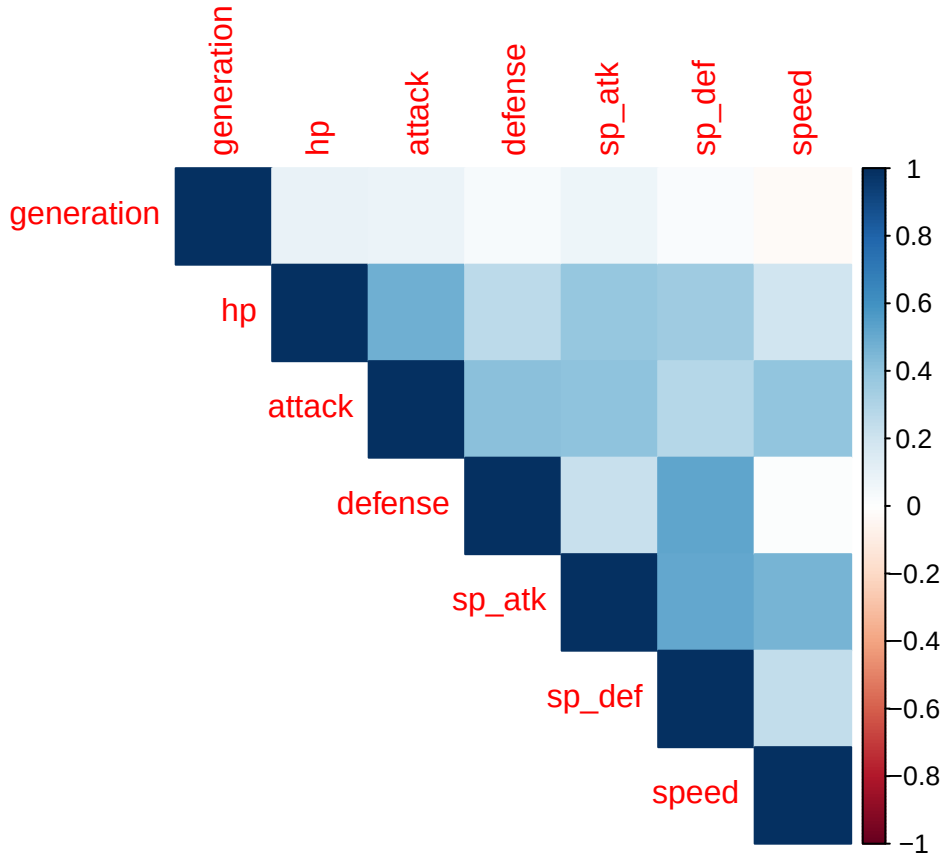
Why do you think doing stratified sampling for cross-validation is useful?

Stratified sampling maintains the same class proportions of the outcome variable in both training set and the validation sets. This can prevent some folds from containing very few observations. Therefore, by using this sampling method, we can get a more reliable result.

Exercise 4

Create a correlation matrix of the training set, using the `corrplot` package. *Note: You can choose how to handle the categorical variables for this plot; justify your decision(s).*

```
num_data <- train %>% select(generation, hp, attack, defense, sp_atk, sp_def, speed)
cor_matrix <- cor(num_data)
corrplot(cor_matrix,
         method = "color",
         type = "upper")
```



For this plot, I only include the numeric variables because correlation measures linear relationship between numeric variables. I removed the categorical variables `type_1` and "legendary' because correlation is meaningless for categorical variables.

What relationships, if any, do you notice?

The plot shows several positive relationships. For example, `hp` and `attack`, `attack` and `defense`, `sp_atk` and `sp_def`, and `sp_atk` and `speed` have a moderate positive correlation. `Defense` and `sp_def` have a strong positive correlation. Variable `generation` did not show a strong relationship with other variables.

Exercise 5

Set up a recipe to predict `type_1` with `legendary`, `generation`, `sp_atk`, `attack`, `speed`, `defense`, `hp`, and `sp_def`.

- Dummy-code `legendary` and `generation`;
- Center and scale all predictors.

```
pok_recipe <- recipe(
  type_1 ~ legendary + generation + sp_atk + attack + speed + defense + hp + sp_def,
  data = train
) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_normalize(all_predictors())
```

Exercise 6

We'll be fitting and tuning an elastic net, tuning `penalty` and `mixture` (use `multinom_reg()` with the `glmnet` engine).

Set up this model and workflow. Create a regular grid for `penalty` and `mixture` with 10 levels each; `mixture` should range from 0 to 1. For this assignment, let `penalty` range from 0.01 to 3 (this is on the `identity_trans()` scale; note that you'll need to specify these values in base 10 otherwise).

```
en_model <- multinom_reg(mixture = tune(), penalty = tune()) %>%
  set_engine("glmnet") %>%
  set_mode("classification")

en_wf <- workflow() %>%
  add_recipe(pok_recipe) %>%
  add_model(en_model)

en_grid <- grid_regular(mixture(range = c(0, 1)),
                        penalty(range = c(-2, log10(3))),
                        levels = 10)
```

Exercise 7

Now set up a random forest model and workflow. Use the `ranger` engine and set `importance = "impurity"`; we'll be tuning `mtry`, `trees`, and `min_n`. Using the documentation for `rand_forest()`, explain in your own words what each of these hyperparameters represent.

mtry represents the number of predictor variables randomly selected at each split of the tree.

trees represents the total number of trees grown in the random forest.

min_n represents the minimum number of observations required in a node for a split to occur.

Create a regular grid with 8 levels each. You can choose plausible ranges for each hyperparameter. Note that `mtry` should not be smaller than 1 or larger than 8. **Explain why neither of those values would make sense.**

```
rf_model <- rand_forest(
  mtry = tune(),
  trees = tune(),
  min_n = tune()
) %>%
  set_engine("ranger", importance = "impurity") %>%
  set_mode("classification")

rf_wf <- workflow() %>%
  add_recipe(pok_recipe) %>%
  add_model(rf_model)

rf_grid <- grid_regular(
  mtry(range = c(1, 8)),
  trees(range = c(100, 800)),
  min_n(range = c(2, 20)),
  levels = 8
)
```

Since the model contains 8 predictor variables, if mtry < 1, there would be no predictors involved in the splitting process. If mtry > 8, it means the model would require more predictor variables exist in the dataset.

For trees I make the range from 100 to 800 because if the tree is too small, we will get noisy result. Therefore, I decide to set the trees from 100 to 800 because I think this will work well for our small dataset.

For min_n, I chose the range from 2 to 20 because in this way the tree will split with enough branch without being overfitting.

What type of model does `mtry = 8` represent?

When mtry $= 8$, it means that all predictor variables are considered at each split. It removes the randomness in the variable selection; therefore, the model becomes a bagged tree model.

Exercise 8

Fit all models to your folded data using `tune_grid()`.

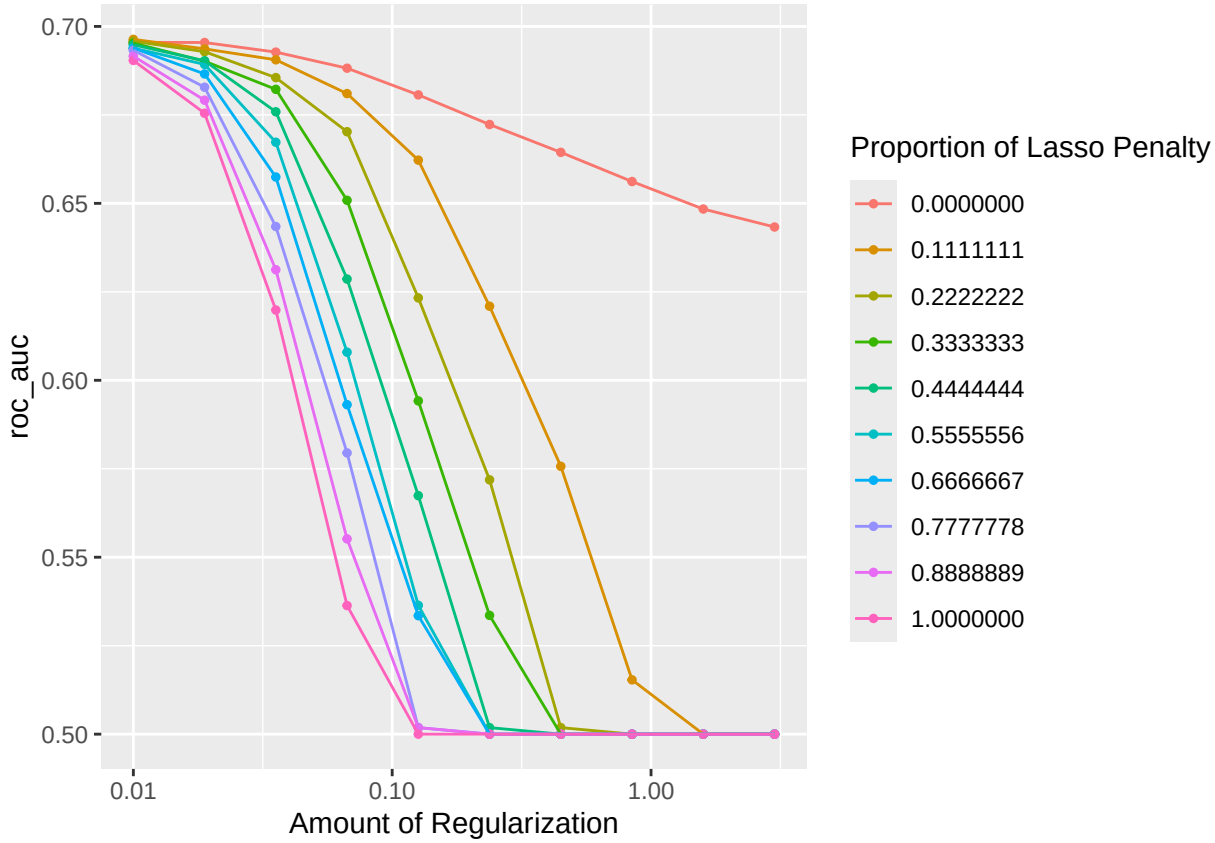
Note: Tuning your random forest model will take a few minutes to run, anywhere from 5 minutes to 15 minutes and up. Consider running your models outside of the .Rmd, storing the results, and loading them in your .Rmd to minimize time to knit. We'll go over how to do this in lecture.

```
en_res <- tune_grid(en_wf,
                    resamples = cv_fold,
                    grid = en_grid,
                    metrics = metric_set(roc_auc))
```

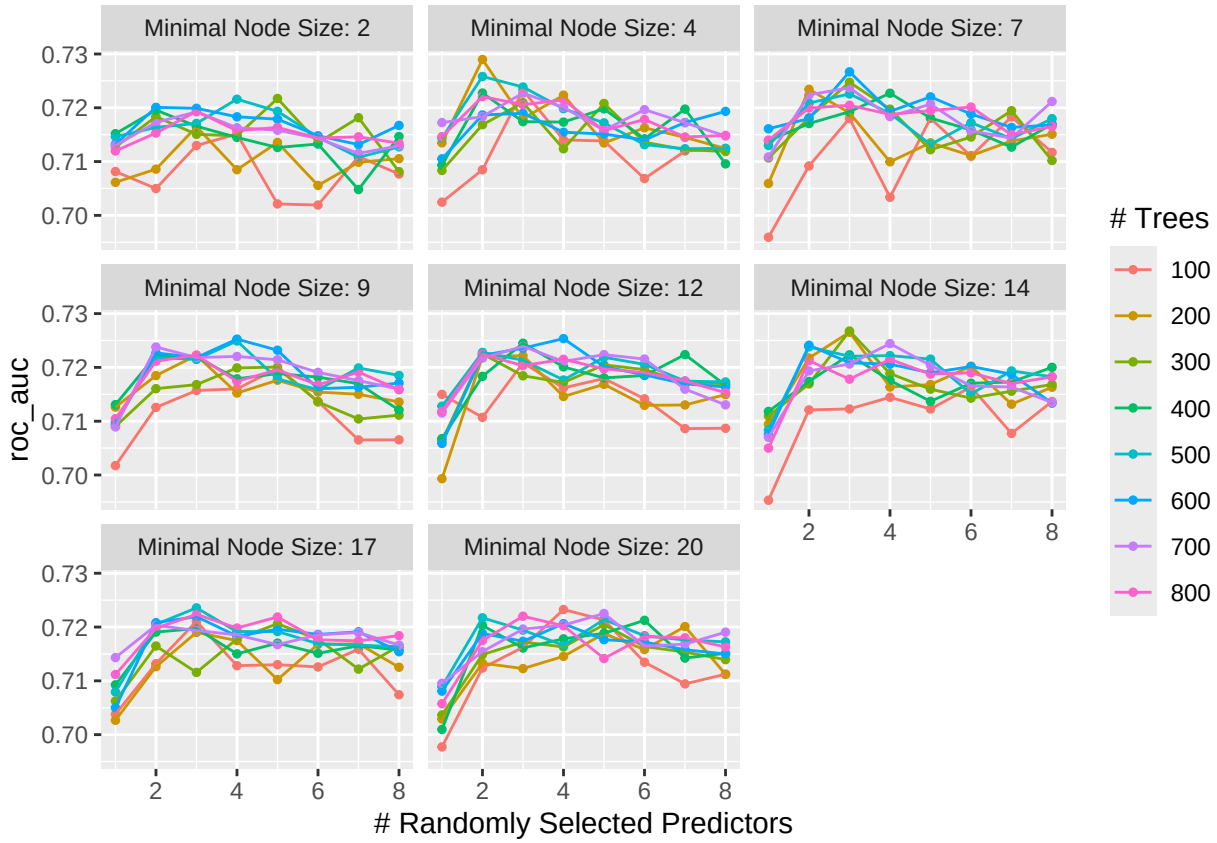
```
rf_res <- tune_grid(rf_wf,
                    resamples = cv_fold,
                    grid = rf_grid,
                    metrics = metric_set(roc_auc))
```

Use `autoplot()` on the results. What do you notice? Do larger or smaller values of `penalty` and `mixture` produce better ROC AUC? What about values of `min_n`, `trees`, and `mtry`?

```
autoplot(en_res)
```




```
autoplot(rf_res)
```




```
collect_metrics(en_res) %>% filter(.metric == "roc_auc") %>%
  select(penalty, mixture, mean, std_err) %>%
  arrange(desc(mean)) %>%
  slice(1)
```

```
## # A tibble: 1 x 4
##   penalty mixture  mean std_err
##     <dbl>   <dbl> <dbl>   <dbl>
## 1    0.01   0.111 0.696  0.0143
```

```
collect_metrics(rf_res) %>% filter(.metric == "roc_auc") %>%
  select(mtry, trees, min_n, mean, std_err) %>%
  arrange(desc(mean)) %>%
  slice(1)
```

```
## # A tibble: 1 x 5
##    mtry trees min_n  mean std_err
##   <int> <int> <int> <dbl>   <dbl>
## 1     2   200     4 0.729  0.0144
```

From the elastic net plot, smaller values of penalty tend to produce better ROC AUC, indicating that less regularization performs better for this dataset. Also, smaller values of mixture also tend to perform better, suggesting that models closer to ridge regression is better than those closer to lasso. For the random forest model, moderate values of mtry (around 2–4) appear to yield the best ROC AUC. Increasing the number of

trees slightly improves stability but does not dramatically change performance. Moderate values of min_n (around 4–9) also tend to perform well.

What elastic net model and what random forest model perform the best on your folded data? (What specific values of the hyperparameters resulted in the optimal ROC AUC?)

The optimal elastic net model used a penalty parameter of 0.01 and a mixture parameter of 0.111. This model achieved a mean ROC AUC of 0.696 with a standard error of 0.0143 on the cross-validated folds. The best random forest model used mtry = 2, 200 trees, and a min_n = 4. This model achieved a mean ROC AUC of 0.729 with a standard error of 0.0144.

Exercise 9

Select your optimal **random forest model** in terms of `roc_auc`. Then fit that model to your training set and evaluate its performance on the testing set.

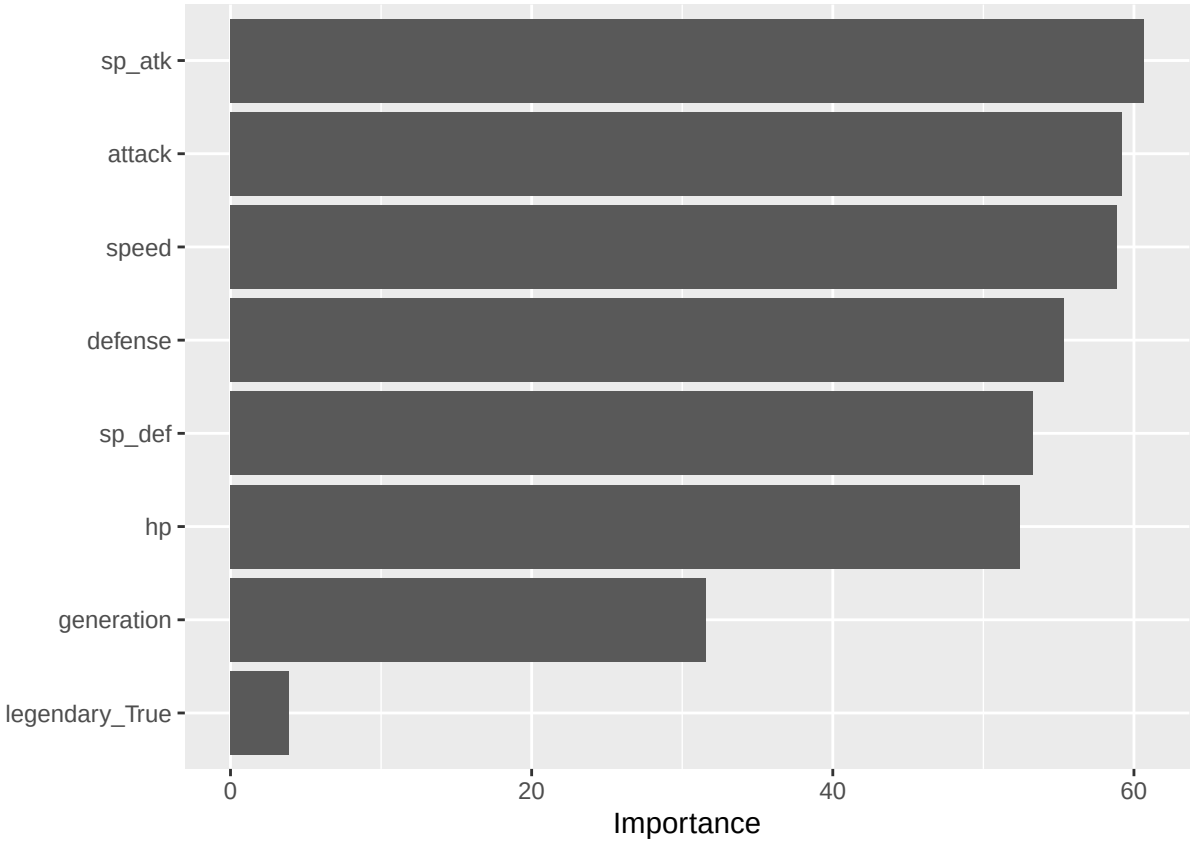
```
best_params <- select_best(rf_res, metric = "roc_auc")
final_wf <- finalize_workflow(rf_wf, best_params)

final_fit <- final_wf %>%
  fit(data = train)
```

Using the **training** set:

- Create a variable importance plot, using `vip()`. *Note that you'll still need to have set `importance = "impurity"` when fitting the model to your entire training set in order for this to work.*

```
final_fit %>%
  extract_fit_engine() %>%
  vip()
```



- What variables were most useful? Which were least useful? Are these results what you expected, or n

The most useful variables are sp_atk, attack, speed, and defense, which have the highest importance values in the random forest model. Variables such as sp_def and hp are also moderately important. Variable such as generation are not very useful, abd legendary_True is the least important variable because it has the smallest importance. The result corresponds to my expectation because some features such as attack, defense and speed determines if a Pokémon is powerful or not. On the contrary, something like generation and legendary do not directly related to their performance.

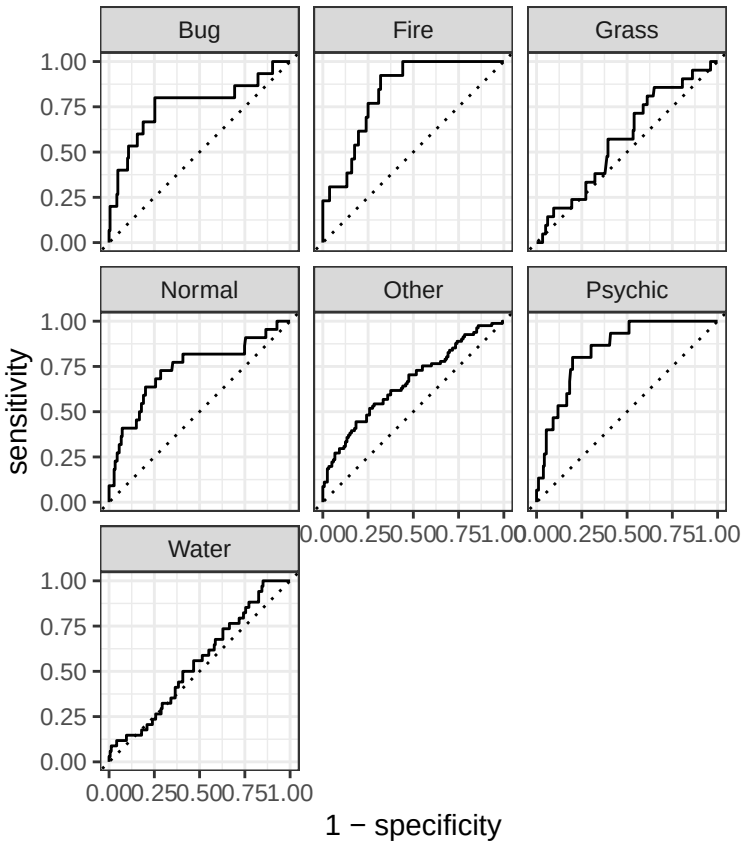
Using the testing set:

```
test_prediction <- augment(final_fit, test)

rf_pred <- predict(final_fit, test, type = "prob") %>%
  bind_cols(predict(final_fit, test)) %>%
  bind_cols(test %>% select(type_1))

prob_cols <- rf_pred %>%
  select(starts_with(".pred_")) %>%
  select(-.pred_class) %>%
  names()

rf_pred %>%
  roc_curve(type_1, all_of(prob_cols)) %>%
  autoplot()
```

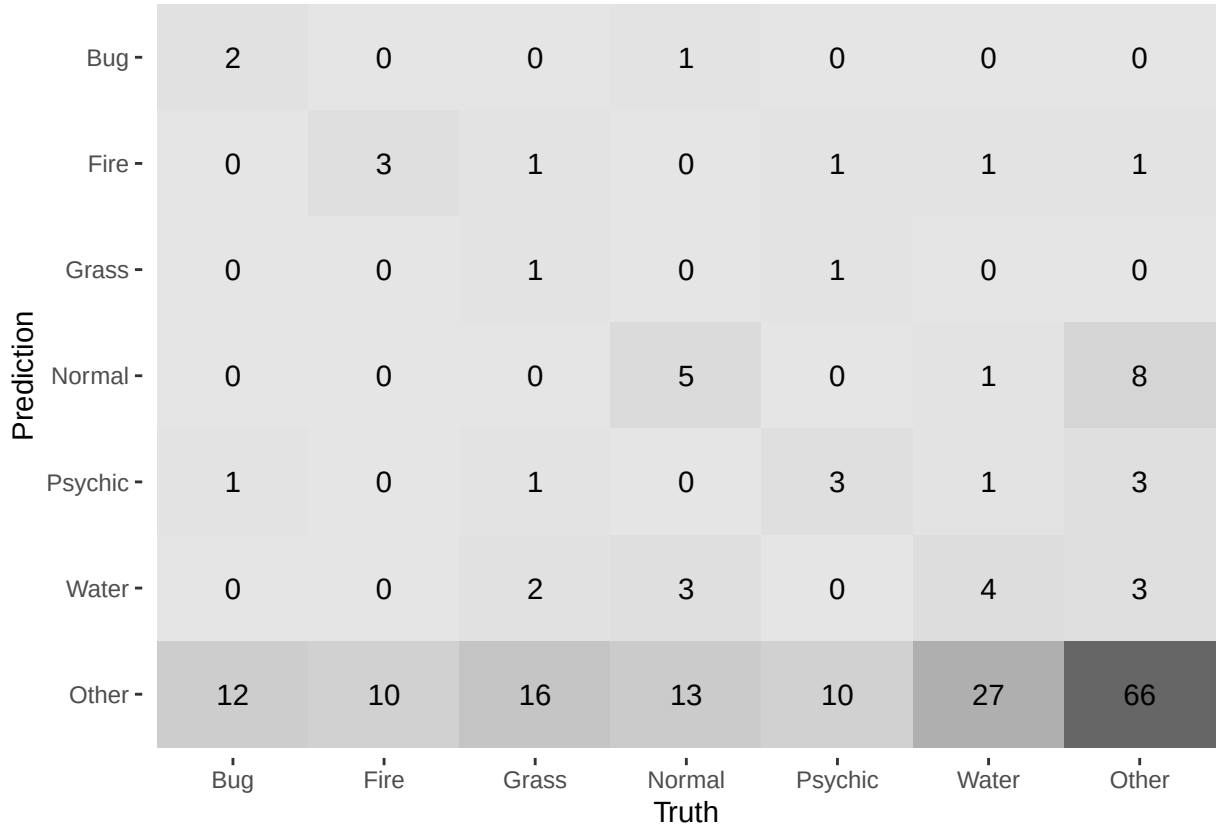



```
cm <- rf_pred %>%
  conf_mat(truth = type_1, estimate = .pred_class)

cm
```

```
##            Truth
## Prediction Bug Fire Grass Normal Psychic Water Other
##     Bug      2    0     0      1       0     0     0
##     Fire     0    3     1      0       1     1     1
##     Grass    0    0     1      0       1     0     0
##     Normal   0    0     0      5       0     1     8
##     Psychic  1    0     1      0       3     1     3
##     Water    0    0     2      3       0     4     3
##     Other   12   10    16     13      10    27    66
```

```
autoplot(cm, type = "heatmap")
```



- Create plots of the different ROC curves, one per level of the outcome variable;
- Make a heat map of the confusion matrix.

The confusion matrix shows that the model performs well for some classes such as Normal and Other, but struggles with smaller classes such as Bug and Grass, which are sometimes misclassified as "Other". This suggests that the model performs better on more frequent categories. The ROC curves show that some classes such as Fire and Psychic achieve relatively high AUC values, while others such as Water perform closer to random guessing.

Exercise 10

How did your best random forest model do on the testing set?

Which Pokemon types is the model best at predicting, and which is it worst at? (Do you have any ideas why this might be?)

The model performs best at predicting Fire, Bug, and Psychic types, as these classes have higher ROC curves and more correct predictions along the confusion matrix diagonal. The model performs worst for Grass and Water, which are often classified as other types by mistake. One possible reason is that the Other class has many observations, and this will cause the model to bias predictions toward that category. This class imbalance likely makes it harder for the model to tell the difference correctly with some of the smaller classes.

For 231 Students

Exercise 11

In the 2020-2021 season, Stephen Curry, an NBA basketball player, made 337 out of 801 three point shot attempts (42.1%). Use bootstrap resampling on a sequence of 337 1's (makes) and 464 0's (misses). For each bootstrap sample, compute and save the sample mean (e.g. bootstrap FG% for the player). Use 1000 bootstrap samples to plot a histogram of those values. Compute the 99% bootstrap confidence interval for Stephen Curry's "true" end-of-season FG% using the quantile function in R. Print the endpoints of this interval.

Exercise 12

Using the `abalone.txt` data from previous assignments, fit and tune a **random forest** model to predict `age`. Use stratified cross-validation and select ranges for `mtry`, `min_n`, and `trees`. Present your results. What was your final chosen model's **RMSE** on your testing set?